Midterm · រៀបចំផែនការសម្រាប់មួយថ្ងៃ

សប្តាហ៍ទី 4 · វគ្គទី 3

90 នាទី · ធ្វើម្នាក់ឯង · ការយល់ដឹង មិនមែនការចងចាំ

អ្វីដែល midterm ពិនិត្យ

- តើអ្នកអាច **អាន** code ហើយពន្យល់ *ហេតុអ្វី* វាដំណើរការបែបនោះបានទេ?
- តើអ្នកអាច **សរសេរ** វិធានមួយ និង function records មួយ ដែលរក្សាការសន្យារបស់វាបានទេ?
- តើអ្នកអាច **test** code ឱ្យចាប់បាន bug ទេ?
- តើអ្នកអាច **វាយតម្លៃ** code ដែល AI សរសេរ ហើយប្រាប់ថាវាធ្វើឱ្យអ្នកណារងគ្រោះបានទេ?
- តើអ្នកអាច **ពន្យល់** ការសម្រេចចិត្តមួយបានទេ?

ស្ថានការណ៍

- អ្នកស្ម័គ្រចិត្តដឹកអាហារក្នុងកន្ត្រកកង់: ម្នាក់ប្រហែល **15 ចំណែក**។
- មុនពេលពួកគេចេញដំណើរ អ្នកសម្របសម្រួលត្រូវដឹង: ចំណែកដែលបានអនុម័ត **ចំនួនអ្នកស្ម័គ្រចិត្តដែលត្រូវការ** និងសំណើដែលនៅ **រង់ចាំ**។

```
{"id": "F003", "food": "Bananas", "portions": 40, "approved": False}
```

ផ្នែកទាំង 6 · 100 ពិន្ទុ

ផ្នែក	កិច្ចការ	ពិន្ទុ
A	អាន code ខ្លីៗ 5 ហើយពន្យល់ *ហេតុអ្វី*	20
B	`volunteers_needed(portions)` ជាមួយ constant	20
C	`plan_day(offers)` return សេចក្តីសង្ខេប	25
D	Test ដែលចាប់ bug ដែលគេដាក់ទុក	15
E	វាយតម្លៃ code ដែល AI សរសេរ	10
F	ការសម្រេចចិត្តរបស់អ្នក + run ស្អាត	10

ផ្នែក D: test ដែលចាប់ bug

```
bridge.catch_bugs([test_normal, test_boundary,
                   test_waiting, test_empty])
```

- Run test របស់អ្នកលើ code **របស់អ្នក** បន្ទាប់មកលើ version 4 ដែលមាន **bug ដាក់ទុក**។
- test ល្អ **ធ្លាក់ ពេល code ខុស**។
- បន្ទាប់មក វាប្តូរការសម្រេចចិត្ត: test ដែលមាន `15` លាក់ទុក នឹងខូច។

ផ្នែក F: ការសម្រេចចិត្តរបស់អ្នក

- កង់ ~15 ដើរ ~8 ម៉ូតូ ~30: ជ្រើសរើសសម្រាប់សាកលវិទ្យាល័យ **របស់អ្នក**។
- ប្តូរ constant → **Runtime → Restart session and run all**។
- អ្វីៗទាំងអស់គួរតែនៅជាប់។ បើមិនជាប់ទេ មានលេខមួយលាក់ទុកនៅកន្លែងណាមួយ។
- កំណត់ត្រាការសម្រេចចិត្ត: ជម្រើសរបស់អ្នក · ហេតុអ្វី (អ្នកណារងផលប៉ះពាល់) · អ្វីបានកើតឡើង ពេលអ្នក run ឡើងវិញ។

ច្បាប់

- **អនុញ្ញាត:** កំណត់ចំណាំរបស់អ្នក notebook ប្រចាំសប្តាហ៍ Python Basics។
- **មិនអនុញ្ញាត:** ឧបករណ៍ AI ការផ្ញើសារ ការចែករំលែក code។
- ដាក់ស្នើ notebook តាមការប្រកាស ក្រោយពេលធ្វើ **Runtime → Restart session and run all**។
- បន្ទាប់មក **ការពិនិត្យផ្ទាល់មាត់ 3 នាទី**: ពន្យល់មួយបន្ទាត់ក្នុង `plan_day` និង test មួយ។

របៀបត្រៀមខ្លួន

- សប្តាហ៍ទី 1: `//` និង `%` (បន្ទុកពេញ និងចំនួនដែលនៅសល់)។
- សប្តាហ៍ទី 2: លំដាប់ branch និង boundary (ព្រំដែន) (`>` ធៀបនឹង `>=`)។
- សប្តាហ៍ទី 3: loop លើ records ឬកសរុប ប្រមូល ID។
- សប្តាហ៍ទី 4: `return` មិនមែន `print`; test សម្រាប់ករណីធម្មតា boundary និង list ទទេ។

សួរមសំណាងល្អ!

អានដោយប្រុងប្រយ័ត្ន · ពន្យល់ *ហេតុអ្វី* · test ករណីនៅតែម · ប្រាប់ថាអ្នកណាងងគ្រោះ